

Unit 5

Unit 5: Relational Database Design

1. Features of Good Relational Design

A relational database is a collection of data items organized as a set of formally-described tables from which data can be accessed efficiently. Before diving into what makes a design "good," it helps to understand what decisions must be made during design:

- Which tables to create
- What columns (attributes) each table will contain
- What the relationships between tables are

A **good relational database design** has the following features:

- **Each table represents a single, well-defined subject.** One table should not store information about two unrelated things.
- **No data item is unnecessarily stored in more than one table.** Minimizing redundancy is the primary goal.
- **All non-prime (non-key) attributes in a table are dependent on the primary key.** Every piece of data in a row should describe the entity identified by the primary key, nothing else.
- **Each table is free from insertion, update, and deletion anomalies.** The design should not create situations where a simple data change requires modifications in multiple places, or where data cannot be stored without unrelated data being present.

2. Problems with Bad Design

A poorly designed schema leads to several serious problems. Consider a situation where the information about instructors and departments, which belongs in two separate tables — `Instructor(ID, Name, Salary)` and `Department(Dept_Name, Building, Budget)` — is combined into a single table `Ins_Dept(ID, Name, Salary, Dept_Name, Building, Budget)`.

This bad design immediately causes:

- **Repetition of Information (Data Redundancy):** The department's building and budget are repeated in every row for every instructor of that department. If a department has 10 instructors, the building and budget data are stored 10 times.
- **Difficulty in updates:** Because the same department information is stored across many tuples, updating the budget of a department requires finding and modifying every tuple for every instructor in that department. Missing even one creates an inconsistency.

- **Inability to represent certain information cleanly:** Some facts cannot be recorded without artificially creating other facts.

These problems are formalized as **data anomalies**, described in detail in Section 8 below.

3. Functional Dependencies

Functional dependencies are the foundation of relational database design. They allow us to formally describe the relationships between attributes in a relation and to distinguish good designs from bad ones.

3.1 Definition

A **functional dependency** is a constraint between two sets of attributes in a relation. Given a relation R , a set of attributes X in R is said to **functionally determine** another attribute Y in R — written as $X \rightarrow Y$ — if and only if each value of X is associated with exactly one value of Y at any given time.

In other words: if two tuples (rows) in the relation have the same values for all attributes (columns) in X , they must also have the same value for Y .

X is called the **determinant** and Y is called the **dependent**.

Example:

Consider an Employee relation:

SSN	Name	Job_Type
12543	Smith	Accountant
23145	John	Accountant
67543	Smith	Engineer

Here, $SSN \rightarrow Name$ is a valid functional dependency because knowing an employee's SSN uniquely determines their name. However, $Name \rightarrow SSN$ is **not** a functional dependency because the name "Smith" appears for two different SSNs.

3.2 Types of Functional Dependencies

Imagine a college database. We have a table that stores student exam results:

`Result(Roll_No, Subject_Code, Subject_Name, Marks, Grade)`

Let's say:

- A student appears for multiple subjects

- Each subject has a fixed name (Subject_Code 'CS101' is always 'Database Systems')
- Marks are what a specific student scored in a specific subject
- Grade is derived from Marks (50–60 = C, 60–75 = B, etc.) — so Grade depends entirely on Marks

The **Primary Key** here is `{Roll_No, Subject_Code}` — because to uniquely identify one result row, you need both the student AND the subject.

Fully Functional Dependency

`{Roll_No, Subject_Code} → Marks`

Marks is **fully** dependent on the composite key because:

- Roll_No alone cannot tell you the marks (a student has different marks in different subjects)
- Subject_Code alone cannot tell you the marks (different students score differently)
- You need **both together** to determine Marks

Removing either attribute from the left side breaks the dependency. That is the definition of a full FD.

Partial Functional Dependency

`{Roll_No, Subject_Code} → Subject_Name`

This is a **partial** dependency because `Subject_Code → Subject_Name` already holds on its own. CS101 is always 'Database Systems' regardless of which student we are talking about. Subject_Name depends only on **part** of the composite key (just Subject_Code), not on the full key.

So Subject_Name does not need Roll_No at all to be determined. That makes it partial.

Transitive Functional Dependency

`{Roll_No, Subject_Code} → Marks → Grade`

Here:

- `{Roll_No, Subject_Code} → Marks` (full FD, as shown above)
- `Marks → Grade` (if you score 72, your grade is B — regardless of which student or subject)

So Grade does not depend on the primary key **directly**. It reaches the primary key only through Marks, which is itself a non-key attribute. This chain — primary key → non-key → another non-key — is called a **transitive dependency**.

The primary key determines Grade, but only *indirectly*, via Marks.

Trivial Functional Dependency

$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Roll_No}$

Roll_No is already **inside** the left side. Of course knowing $\{\text{Roll_No}, \text{Subject_Code}\}$ tells you Roll_No — it's part of what you already know. This dependency is always true no matter what data you have, so it is called trivial. It carries zero new information.

Another trivial example: $\text{Subject_Code} \rightarrow \text{Subject_Code}$

Non-Trivial Functional Dependency

$\text{Roll_No} \rightarrow \text{Student_Name}$

Student_Name is **not** part of Roll_No. Knowing Roll_No genuinely *tells you something new* — the student's name. This is not obvious from the left side alone. That is what makes it non-trivial.

Another: $\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Marks}$ — Marks is not contained in the left side, so this is also non-trivial.

All Types in One Table

FD	Type	Why
$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Marks}$	Fully Functional	Both attributes needed; neither alone works
$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Subject_Name}$	Partial	Subject_Code alone is enough
$\text{Marks} \rightarrow \text{Grade}$ (via primary key)	Transitive	Grade depends on Marks (non-key), not directly on PK
$\{\text{Roll_No}, \text{Subject_Code}\} \rightarrow \text{Roll_No}$	Trivial	Roll_No is already inside the left side
$\text{Roll_No} \rightarrow \text{Student_Name}$	Non-Trivial	Student_Name is genuinely new information

4. Armstrong's Axioms

Armstrong's axioms (developed by William Armstrong in 1974) are a set of **inference rules** used to derive all possible functional dependencies from a given set. They are the basis for computing the closure of a set of functional dependencies. There are six rules in total — three primary and three derived.

Primary Rules

Rule 1: Reflexivity

If Y is a subset of X ($Y \subseteq X$), then $X \rightarrow Y$ holds. Every set of attributes functionally determines any of its subsets.

Example: $\{\text{Name}, \text{Age}\} \rightarrow \text{Name}$ is always true because $\text{Name} \subseteq \{\text{Name}, \text{Age}\}$.

Rule 2: Augmentation

If $X \rightarrow Y$ holds, and W is any set of attributes, then $WX \rightarrow WY$ also holds. Adding the same attributes to both sides of a functional dependency preserves it.

Example: If $A \rightarrow B$, then $AC \rightarrow BC$ also holds.

Rule 3: Transitivity

If $X \rightarrow Y$ and $Y \rightarrow Z$ both hold, then $X \rightarrow Z$ also holds.

Example: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$.

Derived Rules

Rule 4: Union

If $X \rightarrow Y$ and $X \rightarrow Z$ both hold, then $X \rightarrow YZ$ holds. If one determinant determines multiple things separately, it determines them together.

Example: If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow BC$.

Rule 5: Decomposition

If $X \rightarrow YZ$ holds, then $X \rightarrow Y$ and $X \rightarrow Z$ both hold separately. The reverse of the union rule.

Example: If $A \rightarrow BC$, then $A \rightarrow B$ and $A \rightarrow C$.

Rule 6: Pseudo-Transitivity (Composition)

If $X \rightarrow Y$ and $WY \rightarrow Z$ hold, then $WX \rightarrow Z$ also holds.

Example: If $A \rightarrow B$ and $CB \rightarrow D$, then $CA \rightarrow D$.

Self-Determination

Every attribute (or set of attributes) determines itself: $X \rightarrow X$. This is a direct result of the Reflexivity rule and is always trivially true.

Example: $A \rightarrow A$, $\{A, B\} \rightarrow \{A, B\}$.

Worked Example Using Armstrong's Axioms

Given: $R = (A, B, C, G, H, I)$ and $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

- **Proving $A \rightarrow H$:** Since $A \rightarrow B$ and $B \rightarrow H$, by Transitivity: $A \rightarrow H$.
- **Proving $CG \rightarrow HI$:** Since $CG \rightarrow H$ and $CG \rightarrow I$, by Union: $CG \rightarrow HI$.

- **Proving $AG \rightarrow I$:** Since $A \rightarrow C$ and $CG \rightarrow I$, by Pseudo-Transitivity: $AG \rightarrow I$. Alternatively, apply Augmentation to $A \rightarrow C$ to get $AG \rightarrow CG$, then Transitivity with $CG \rightarrow I$ to get $AG \rightarrow I$.
-

5. Closure of Functional Dependencies (F^+)

The **closure of a set of functional dependencies F** , denoted F^+ , is the set of **all functional dependencies that are logically implied by F** — meaning all FDs that can be derived from F using Armstrong's Axioms.

In simpler terms: if F is the set of FDs explicitly given for a relation R , then F^+ contains not just those FDs, but every FD that can be inferred from them by applying the axioms repeatedly.

Why compute F^+ ?

- To identify all FDs that must hold in any valid instance of the relation
- To determine whether a decomposition preserves all the original constraints

Steps to Compute F^+

1. Start with $F^+ = F$ (the original set of FDs)
 2. Apply the Reflexivity rule: add all trivial FDs
 3. Apply the Augmentation rule to every FD in F^+ to generate new FDs; add them to F^+
 4. Apply the Transitivity rule to pairs of FDs in F^+ ; add any new FDs to F^+
 5. Repeat steps 2–4 until no new FDs can be added
-

6. Closure of Attribute Sets (X^+)

The **closure of a set of attributes X under a set of functional dependencies F** , denoted X^+ , is the set of **all attributes that are functionally determined by X** using the FDs in F .

This is a practical and efficient way to work with functional dependencies without computing the entire F^+ .

Main uses of attribute closure:

- To find all attributes determined by a given set
- To check whether a given set is a **super key** (if $X^+ = \text{all attributes of } R$, then X is a super key)
- To identify **candidate keys**

Finding Super Keys and Candidate Keys Using Closure

- **Super key:** X is a super key if X^+ contains all attributes of R .
- **Candidate key:** X is a candidate key if $X^+ = \text{all attributes of } R$, AND no proper subset of X has this property (i.e., X is minimal).

Reminder: Every candidate key is a super key, but not every super key is a candidate key. A super key may contain extra, redundant attributes. A candidate key is the minimal version — remove any attribute from it and it can no longer uniquely identify a row.

Worked Example:

$R = \{A, B, C, D, E\}$, $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

Compute A^+ :

- Start: $A^+ = \{A\}$
- Apply $A \rightarrow BC$: $A^+ = \{A, B, C\}$
- Apply $B \rightarrow D$: $A^+ = \{A, B, C, D\}$
- Apply $CD \rightarrow E$: $A^+ = \{A, B, C, D, E\}$
- $A^+ =$ all attributes $\rightarrow$ **A is a super key**

Compute CD^+ :

- Start: $CD^+ = \{C, D\}$
- Apply $CD \rightarrow E$: $CD^+ = \{C, D, E\}$
- Apply $E \rightarrow A$: $CD^+ = \{A, C, D, E\}$
- Apply $A \rightarrow BC$: $CD^+ = \{A, B, C, D, E\}$
- $CD^+ =$ all attributes $\rightarrow$ **CD is a super key**

Compute E^+ :

- Start: $E^+ = \{E\}$
- Apply $E \rightarrow A$: $E^+ = \{A, E\}$
- Apply $A \rightarrow BC$: $E^+ = \{A, B, C, E\}$
- Apply $B \rightarrow D$: $E^+ = \{A, B, C, D, E\}$
- $E^+ =$ all attributes $\rightarrow$ **E is a super key**

Checking minimality:

- A is a single attribute $\rightarrow$ cannot reduce further $\rightarrow$ **A is a candidate key**
- CD: check C^+ and D^+ separately — if neither alone covers all attributes, CD is minimal $\rightarrow$ **CD is a candidate key**
- E is a single attribute $\rightarrow$ **E is a candidate key**

7. Canonical Cover

The Problem It Solves

Every time the database is updated (insert, update, delete), the system must check that the update does not violate any functional dependency in F . If F contains many redundant or overlapping dependencies, this checking becomes unnecessarily expensive. The system might be checking the same constraint multiple times in different forms.

A **canonical cover** (also called a minimal cover) is a **simplified, minimal equivalent of F** — it contains no redundant FDs and no redundant attributes within FDs, but implies exactly the same constraints as F . Checking the canonical cover is enough; we don't need to check the full F .

Extraneous Attributes

Before computing the canonical cover, we need to understand **extraneous attributes**. An attribute is extraneous in a functional dependency if it can be safely removed without changing what the overall set F implies.

There are two places an attribute can be extraneous:

Extraneous on the Left Side (LHS): Attribute A is extraneous in $X \rightarrow Y$ if removing A from X still allows Y to be derived from the remaining FDs. In other words, the attribute was not needed in the first place.

Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$

- Is B extraneous in $AB \rightarrow C$?
- Compute A^+ using F : $A^+ = \{A, C\}$ — A alone already determines C .
- So B is not needed. B is extraneous in $AB \rightarrow C$.
- F simplifies to: $\{A \rightarrow C\}$ ($AB \rightarrow C$ is replaced by $A \rightarrow C$, which was already there)

Extraneous on the Right Side (RHS): Attribute C is extraneous in $X \rightarrow YC$ if removing C from the RHS still allows C to be derived through other FDs.

Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$

- Is C extraneous in $AB \rightarrow CD$?
- Even if we remove C from $AB \rightarrow CD$, we still have $A \rightarrow C$, and A is a subset of AB , so AB still determines C through $A \rightarrow C$.
- Therefore C is extraneous in $AB \rightarrow CD$.
- F simplifies to: $\{A \rightarrow C, AB \rightarrow D\}$

Steps to Compute the Canonical Cover (F_c)

Step 1: Decompose all FDs so that each has exactly one attribute on the RHS.

Replace every FD of the form $X \rightarrow A_1A_2...A_k$ with separate FDs:

$X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_k$

Example: $A \rightarrow BC$ becomes $A \rightarrow B$ and $A \rightarrow C$.

Step 2: Eliminate extraneous attributes from the LHS of each FD.

For every FD $X \rightarrow A$ in F , check if any attribute in X is extraneous. If removing an attribute from X still allows A to be derived (using the remaining FDs), remove it.

Step 3: Eliminate redundant FDs.

For every FD $X \rightarrow A$ in F , temporarily remove it from F and compute X^+ using the remaining FDs. If A is still in X^+ , then $X \rightarrow A$ is redundant — it can be derived without being explicitly stated. Remove it permanently.

Step 4: Recombine (optional).

If multiple FDs share the same LHS after the above steps, merge their RHS back together:

$X \rightarrow A_1, X \rightarrow A_2$ becomes $X \rightarrow A_1A_2$

The result is the canonical cover F_c .

Worked Example

Given: $F = \{A \rightarrow C, AB \rightarrow C, AB \rightarrow D\}$

Step 1: Decompose RHS

- $A \rightarrow C$
- $AB \rightarrow C$
- $AB \rightarrow D$

(Already decomposed — each FD has one attribute on RHS)

Step 2: Eliminate extraneous LHS attributes

Check $AB \rightarrow C$: Is A extraneous in $AB \rightarrow C$?

- Compute B^+ using F : B alone appears in no FD as a determinant $\rightarrow B^+ = \{B\}$. B does not determine C . So A is not extraneous here.

Check $AB \rightarrow C$: Is B extraneous in $AB \rightarrow C$?

- Compute A^+ using F : $A \rightarrow C$, so $A^+ = \{A, C\}$. A alone determines C . So B IS extraneous in $AB \rightarrow C$.
- Remove B : $AB \rightarrow C$ becomes $A \rightarrow C$. But $A \rightarrow C$ already exists in F , so $AB \rightarrow C$ is simply dropped.

F is now: $\{A \rightarrow C, AB \rightarrow D\}$

Check $AB \rightarrow D$: Is A extraneous?

- Compute $B^+ = \{B\}$ (B determines nothing). B alone does not determine D . So A is not extraneous.

Check $AB \rightarrow D$: Is B extraneous?

- Compute $A^+ = \{A, C\}$. A alone does not determine D. So B is not extraneous.

Step 3: Eliminate redundant FDs

Check $A \rightarrow C$: Remove it temporarily. $F = \{AB \rightarrow D\}$. Compute $A^+ = \{A\}$. C is not in A^+ . So $A \rightarrow C$ is NOT redundant — keep it.

Check $AB \rightarrow D$: Remove it temporarily. $F = \{A \rightarrow C\}$. Compute $AB^+ = \{A, B, C\}$. D is not in AB^+ . So $AB \rightarrow D$ is NOT redundant — keep it.

Canonical Cover $F_c = \{A \rightarrow C, AB \rightarrow D\}$

8. Data Redundancy

Data redundancy means storing the same piece of information in more than one place in the database. It is the root cause of most database design problems.

Example: If the department name, building, and budget are stored in every employee record (instead of in a separate Department table), and a department has 50 employees, the department information is stored 50 times. Now if the building name changes, all 50 rows must be updated.

Redundancy causes:

- Wasted storage space
 - Inconsistencies when updates are not applied uniformly
 - Data anomalies (described below)
-

9. Data Anomalies

Data anomalies are problems that arise during data manipulation operations (insert, update, delete) as a direct result of redundancy and poor design. There are three types:

9.1 Update Anomaly (Modification Anomaly)

An update anomaly occurs when changing one piece of data requires updating multiple rows. If even one row is missed, the database becomes inconsistent — different rows contain different values for the same real-world fact.

Example: In the combined `Ins_Dept` table:

```
Ins_Dept(ID, Name, Salary, Dept_Name, Building, Budget)
```

If the budget of the "Physics" department changes, every row for every Physics instructor must be updated. If some rows are missed, the database will have contradictory budget values for the same

department.

9.2 Insertion Anomaly

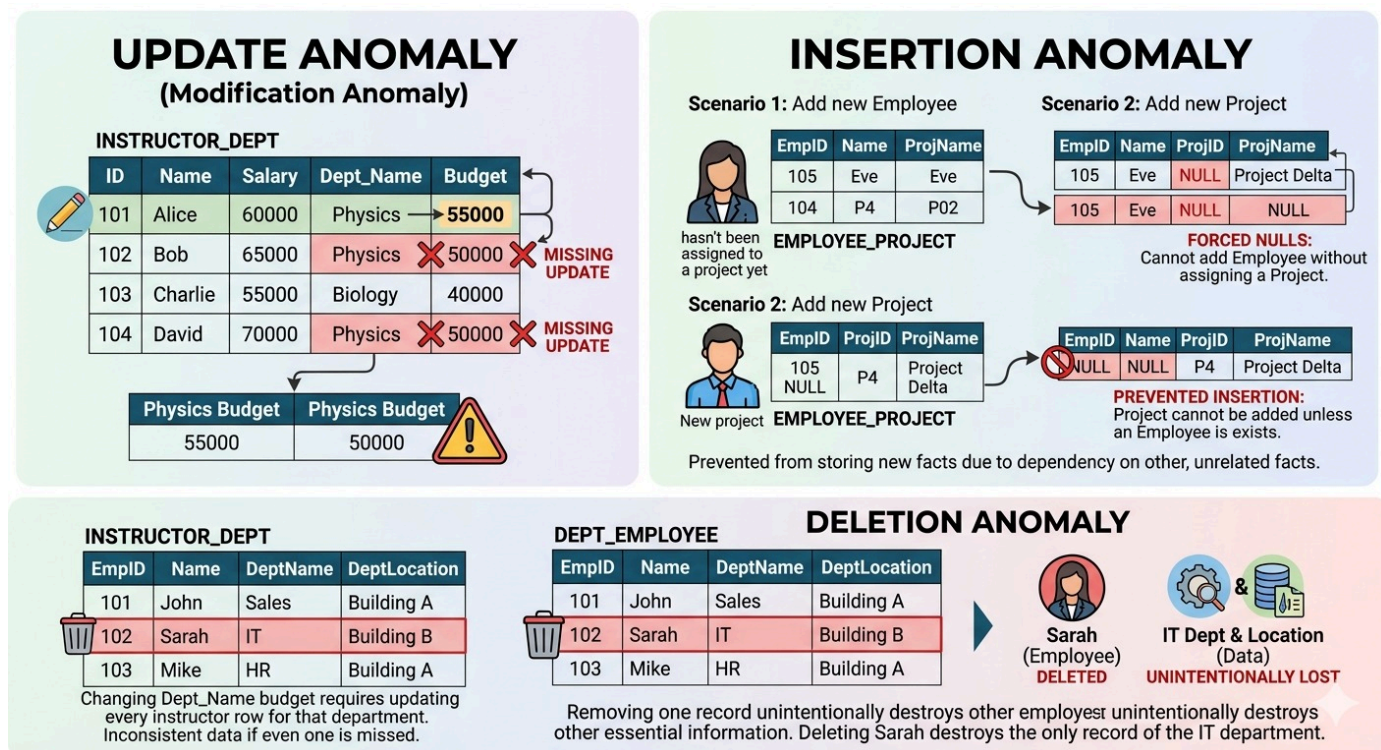
An insertion anomaly occurs when certain facts cannot be recorded unless other, unrelated facts are also present. You are forced to store dummy or NULL data just to satisfy the table structure.

Example: In a bad design where employee data and project data are merged, you cannot add a new employee to the database unless they are already assigned to a project. If an employee has just been hired but not yet assigned, you would need to create a phantom project just to complete the row. Similarly, you cannot record a new department unless at least one employee exists in that department.

9.3 Deletion Anomaly

A deletion anomaly occurs when deleting one piece of data unintentionally destroys other, unrelated information.

Example: In a merged table, if only one employee is assigned to a particular project, deleting that employee's record also deletes the only record of that project — the project information is permanently lost even though the project itself still exists. To prevent this, you would have to keep a fictitious employee record just to preserve the project data.



10. Decomposition

When a table has too many attributes crammed together (causing redundancy and anomalies), we **split it into smaller tables**. This splitting is called decomposition.

For example, if R has attributes (A, B, C, D, E), we might split it into R1(A, B, C) and R2(A, D, E). The key rule is that every attribute from the original table must appear in at least one of the smaller tables —

nothing gets lost.

But splitting carelessly creates new problems. A good decomposition must satisfy two conditions.

Condition 1 — Lossless Join

When you split a table and later **join the pieces back together**, you should get the exact original table — nothing more, nothing less.

If joining gives you **extra rows that were never in the original data**, the decomposition is called **lossy**. Those extra rows are fake rows that got created because of a bad split.

Example of Lossless Decomposition:

Original table R:

Student	Subject	Teacher
Alice	Maths	Mr. Roy
Bob	Science	Ms. Rao
Alice	Science	Ms. Rao

Split into R1(Student, Subject) and R2(Subject, Teacher):

R1:

Student	Subject
Alice	Maths
Bob	Science
Alice	Science

R2:

Subject	Teacher
Maths	Mr. Roy
Science	Ms. Rao

Joining R1 and R2 back on Subject gives:

Student	Subject	Teacher
Alice	Maths	Mr. Roy
Bob	Science	Ms. Rao
Alice	Science	Ms. Rao

This is exactly the original table — no extra rows. This is a **lossless decomposition**.

Example of Lossy Decomposition:

Original table R:

Student	Subject	Teacher
Alice	Maths	Mr. Roy
Bob	Maths	Ms. Rao

Split into R1(Student, Subject) and R2(Subject, Teacher):

R1:

Student	Subject
Alice	Maths
Bob	Maths

R2:

Subject	Teacher
Maths	Mr. Roy
Maths	Ms. Rao

Joining R1 and R2 back on Subject:

Student	Subject	Teacher
Alice	Maths	Mr. Roy
Alice	Maths	Ms. Rao ← Fake! Alice only had Mr. Roy as a teacher
Bob	Maths	Mr. Roy ← Fake! Bob only had Ms. Rao as a teacher

Student	Subject	Teacher
Bob	Maths	Ms. Rao

The join on Subject is ambiguous — the same subject appears with multiple teachers, so every student gets paired with every teacher. We can no longer tell which teacher actually taught which student. This is a **lossy decomposition**.

How to check if a decomposition is lossless:

Find the common attributes between R1 and R2 (i.e., $R1 \cap R2$). If those common attributes form a **super key for either R1 or R2**, the decomposition is lossless.

In the lossy example above, $R1 \cap R2 = \{\text{Subject}\}$. Subject is not a key for either table — so the decomposition is lossy. In the lossless example, $R1 \cap R2 = \{\text{Subject}\}$, and Subject IS a key for R2 (each subject maps to exactly one teacher) — so it is lossless.

Condition 2 — Dependency Preservation

Every functional dependency that existed in the original table should still be **checkable in at least one of the smaller tables**, without having to join anything.

FDs are rules the data must follow. If after splitting, some rule can only be verified by joining two tables back together, enforcing that rule becomes very expensive. We want every rule to be locally checkable in one table.

Example:

R(Emp_ID, Dept_ID, Dept_Name), FDs: `Emp_ID → Dept_ID` and `Dept_ID → Dept_Name`

Split into:

- R1(Emp_ID, Dept_ID) — contains `Emp_ID → Dept_ID` ✓
- R2(Dept_ID, Dept_Name) — contains `Dept_ID → Dept_Name` ✓

Both FDs are preserved. This is **dependency-preserving**.

Now suppose we split into R1(Emp_ID, Dept_Name) and R2(Dept_ID, Dept_Name). The FD `Emp_ID → Dept_ID` cannot be verified in either table alone — Dept_ID is missing from R1. This is **not dependency-preserving**.

In short:

- **Lossless** = joining the pieces gives back the exact original table (no fake rows)
- **Dependency-preserving** = every FD rule can be checked in one of the smaller tables

A good decomposition satisfies **both**.

11. Normalization

Normalization is a systematic, step-by-step process of structuring (decomposing) a relation to minimize data redundancy and eliminate data anomalies, while preserving all the data and the functional dependencies. Each step of normalization moves the relation to a higher **normal form**.

Goals of Normalization:

- Ensure data integrity
- Prevent data redundancy
- Avoid data anomalies (update, insertion, deletion)
- Ensure each table represents a single well-defined subject

Important Terminology:

- **Prime attribute (key attribute):** Any attribute that is part of at least one candidate key
- **Non-prime attribute (non-key attribute):** An attribute that is not part of any candidate key

All normal forms are **cumulative** — each one builds on top of the previous. Higher normal forms are always better than lower ones from a design standpoint.

12. First Normal Form (1NF)

Theory

A relation is in **First Normal Form (1NF)** if:

1. **All attribute values are atomic** — each cell contains exactly one indivisible value. No comma-separated lists, no sets of values inside a single cell.
2. **There are no repeating groups** — the same type of information is not stored across multiple columns of the same row (e.g., Phone1, Phone2, Phone3).
3. **A primary key is defined** — every row is uniquely identifiable.
4. **All attributes depend on the primary key.**

Applying to the Example

When the restaurant first starts recording employee data, they create this rough table:

employee_id	name	job_code	job	state_code	home_state
E001	Alice	J01, J02	Chef, Waiter	26	Michigan
E002	Bob	J02, J03	Waiter, Bartender	56	Wyoming

employee_id	name	job_code	job	state_code	home_state
E003	Alice	J01	Chef	56	Wyoming

This table **violates 1NF** because:

- `job_code` contains multiple values in a single cell (J01, J02) — not atomic
- `job` contains multiple values in a single cell (Chef, Waiter) — not atomic

To convert to 1NF, each job gets its own row. Every cell must hold exactly one value:

employee_id	name	job_code	job	state_code	home_state
E001	Alice	J01	Chef	26	Michigan
E001	Alice	J02	Waiter	26	Michigan
E002	Bob	J02	Waiter	56	Wyoming
E002	Bob	J03	Bartender	56	Wyoming
E003	Alice	J01	Chef	56	Wyoming

Now every cell is atomic. A **composite primary key** of `(employee_id, job_code)` is defined, because one employee can hold multiple job codes and one job code can be held by multiple employees — neither alone is unique, but together they identify each row.

The table is now in 1NF.

However, notice that `name`, `state_code`, and `home_state` are now repeated for every row of the same employee. This is because those attributes depend only on `employee_id`, not on the full composite key `(employee_id, job_code)`. This is a **partial dependency** — the problem that 2NF addresses.

13. Second Normal Form (2NF)

Theory

A relation is in **Second Normal Form (2NF)** if:

1. It is in **1NF**, AND
2. **Every non-prime attribute is fully functionally dependent on the entire primary key** — there are no partial dependencies.

A **partial dependency** exists when a non-key attribute depends on only *part* of the composite primary key, not on all of it.

Note: If the primary key is a single column (not composite), the table is automatically in 2NF — you cannot have a "part" of a single-column key.

How to fix partial dependencies:

- For each part of the composite key that alone determines some non-key attributes, create a new table with that part as the primary key.
- Move those dependent attributes into the new table.
- Keep only the attributes that truly depend on the full composite key in the original table.

Applying to the Example

In our 1NF table, the composite PK is `(employee_id, job_code)`. Checking every non-key attribute:

- `name` — depends only on `employee_id` (partial dependency X)
- `state_code` — depends only on `employee_id` (partial dependency X)
- `home_state` — depends only on `employee_id` (partial dependency X)
- `job` — depends only on `job_code` (partial dependency X)

None of the non-key attributes depend on the *full* composite key. We fix this by splitting into three tables:

employee_roles table (keeps only the full-key relationship):

employee_id	job_code
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

employees table (attributes depending only on `employee_id`):

employee_id	name	state_code	home_state
E001	Alice	26	Michigan
E002	Bob	56	Wyoming
E003	Alice	56	Wyoming

jobs table (attributes depending only on `job_code`):

job_code	job
J01	Chef

job_code	job
J02	Waiter
J03	Bartender

The tables are now in 2NF — no partial dependencies remain. But look at the `employees` table. If you know `state_code = 26`, you know `home_state = Michigan`. So `home_state` depends on `state_code`, which is a non-key attribute — not on the primary key `employee_id` directly. This is a **transitive dependency** — the problem that 3NF addresses.

14. Third Normal Form (3NF)

Theory

A relation is in **Third Normal Form (3NF)** if:

1. It is in **2NF**, AND
2. **There are no transitive functional dependencies** — no non-prime attribute depends on another non-prime attribute.

Every non-key attribute must depend **directly and only** on the primary key — not indirectly through another non-key attribute.

A transitive dependency looks like this: `PK → A → B`, where A and B are both non-key attributes. B reaches the primary key only through A.

How to fix transitive dependencies:

- Identify the non-key attribute (A) that acts as an intermediate determinant.
- Create a new table with A as the primary key and B as a non-key attribute.
- Remove B from the original table but keep A as a foreign key to maintain the link.

Applying to the Example

In the `employees` table (PK = `employee_id`):

- `employee_id → state_code` ✓ (direct dependency on PK)
- `state_code → home_state` ← this is the problem. `home_state` depends on `state_code`, which is itself a non-key attribute.
- So `employee_id → state_code → home_state` is a **transitive dependency**.

We fix this by extracting `state_code` and `home_state` into their own table:

`employees` table (transitive dependency removed):

employee_id	name	state_code
E001	Alice	26
E002	Bob	56
E003	Alice	56

`state_code` stays as a foreign key in the employees table, linking to the new states table.

`states` table (new table for the transitive dependency):

state_code	home_state
26	Michigan
56	Wyoming

`employee_roles` table (unchanged):

employee_id	job_code
E001	J01
E001	J02
E002	J02
E002	J03
E003	J01

`jobs` table (unchanged):

job_code	job
J01	Chef
J02	Waiter
J03	Bartender

The database is now in 3NF. Every non-key attribute in every table depends directly and only on that table's primary key.

15. Boyce-Codd Normal Form (BCNF)

Theory

BCNF is a stricter version of 3NF. A relation is in BCNF if:

1. It is in **3NF**, AND
2. For **every non-trivial functional dependency** $X \rightarrow Y$, **X must be a super key**.

The difference between 3NF and BCNF is subtle. 3NF allows a non-super-key to be a determinant, as long as the dependent attribute (Y) is a prime attribute. BCNF does not allow this — every determinant must be a super key, no exceptions.

This situation arises when a relation has **more than one overlapping candidate key**.

BCNF Rule — for every FD $X \rightarrow Y$:

- If it is a trivial FD ($Y \subseteq X$) $\rightarrow$ fine
- If X is a super key $\rightarrow$ fine
- Otherwise $\rightarrow$ **not in BCNF**, must decompose

Extending the Example

Our current 3NF design is already in BCNF for all tables. Let us introduce a new scenario to show when 3NF passes but BCNF fails.

Suppose we add a business rule: each employee can have multiple skills, each trainer teaches only one skill, but a skill can have multiple trainers. We store this in:

training (employee_id, skill, trainer)

employee_id	skill	trainer
E001	Cooking	Mr. A
E001	Plating	Mr. B
E002	Cooking	Mr. C
E003	Cooking	Mr. A

Functional dependencies:

- (employee_id, skill) $\rightarrow$ trainer — given an employee and a skill, we know the trainer
- trainer $\rightarrow$ skill — each trainer teaches only one skill

Candidate keys:

- (employee_id, skill) ✓
- (employee_id, trainer) ✓

All attributes are prime (each appears in a candidate key). So there are no transitive dependencies involving non-prime attributes — the table **is in 3NF**.

But look at `trainer → skill`. Here `trainer` is **not** a super key (trainer alone doesn't identify an employee), yet it determines `skill`. 3NF allows this because `skill` is a prime attribute. **BCNF does not allow it**.

The table is in 3NF but NOT in BCNF.

Decomposing to BCNF:

`trainer_skill` table:

trainer	skill
Mr. A	Cooking
Mr. B	Plating
Mr. C	Cooking

PK: `trainer` — trainer is now a super key here ✓

`employee_trainer` table:

employee_id	trainer
E001	Mr. A
E001	Mr. B
E002	Mr. C
E003	Mr. A

PK: `(employee_id, trainer)`

Now every determinant in both tables is a super key. **Both tables are in BCNF**.

Trade-off: Decomposing to BCNF sometimes causes loss of dependency preservation. The FD `(employee_id, skill) → trainer` can no longer be checked in a single table — you would need to join both tables to verify it. When BCNF cannot be achieved without losing dependency preservation, we accept 3NF as the final design.

16. Summary

Normal Form	Condition	What it Eliminates
1NF	Atomic values, primary key defined, no repeating groups	Non-atomic values, repeating groups
2NF	In 1NF + no partial dependencies	Non-key attributes depending on part of composite key
3NF	In 2NF + no transitive dependencies	Non-key attributes depending on other non-key attributes
BCNF	In 3NF + every determinant is a super key	Any non-super-key determinant, even if dependent is prime

Step-by-step ladder:

Unnormalized

↓ **Ensure** atomic values, define PK, **remove** repeating groups

1NF

↓ **Remove partial** dependencies → split **on** parts **of** composite key

2NF

↓ **Remove** transitive dependencies → split **on** non-key determinants

3NF

↓ **Ensure** every determinant **is** a super key → split overlapping candidate key

violations

BCNF

Reference: <https://www.freecodecamp.org/news/database-normalization-1nf-2nf-3nf-table-examples/>

<https://www.digitalocean.com/community/tutorials/database-normalization>